QA TEST REPORT

Web Application — Functional & Security Validation

Target Platform	[REDACTED DOMAIN] (online classifieds / neighborhood marketplace platform)
Test Type	Black-Box Functional & Security QA Testing
Test Date	June 12, 2026
Test Cases Executed	8
Document Status	Final Report

1. Objective

This report documents a structured QA test pass against a live web application, covering both functional API behavior and security-relevant edge cases (authentication, session management, authorization, and input validation). Each test case below is written to be independently reproducible: a tester following the documented preconditions, steps, and test data should be able to re-run the case and arrive at the same actual result. Findings are classified using a Pass / Fail / Observation status model, with a risk rating attached to any non-passing case.

2. Test Methodology

- **Approach:** black-box testing — no source code or internal documentation access; all test cases were derived from observing the live application as an external user/attacker would.
- **Tooling:** Nmap (network/port discovery), Burp Suite — Repeater and Intruder (request manipulation and automated submission), Postman/cURL (API verification), and manual browser-based exploratory testing.
- **Test design techniques:** boundary value and negative testing for input validation cases; equivalence partitioning for authenticated vs. unauthenticated and authorized vs. unauthorized request scenarios; exploratory testing guided by the OWASP API Security Top 10 (Broken Object Level Authorization, Excessive Data Exposure, Mass Assignment, Security Misconfiguration).
- **Classification model:** each test case is scored PASS, FAIL, or OBSERVATION. FAIL/OBSERVATION cases additionally receive a risk rating (Low / Medium / High / Critical) to support prioritization.

3. Scope

In Scope

- Listing creation/update API and its response payloads
- File/image upload handling (SVG ingestion pipeline)

- Session and cookie management
- Object-level authorization on update/delete operations
- Mass-assignment protection on profile update endpoints
- Admin authentication throttling
- Perimeter network exposure and HTTP/HTTPS transport enforcement

Out of Scope

- Source code review
- Internal/corporate network testing
- Social engineering or physical security testing

4. Test Environment

Environment: Production (live), external black-box view — no staging or source access
Client: Latest Chrome/Firefox, routed through Burp Suite proxy
Network: Public internet, no VPN or internal network access

5. Test Case Summary

Of the 8 test cases executed, 4 passed, 3 failed, and 1 produced an observation warranting a recommended enhancement.

ID	Test Case	Category	Status	Risk
TC-01	Perimeter & Service Exposure Scan	Infrastructure	PASS	—
TC-02	Sensitive Data Exposure in API Response	API / Security	FAIL	Medium
TC-03	Stored Content Injection via SVG Upload	Input Validation	FAIL	Medium
TC-04	Object-Level Authorization (IDOR/BOLA) on Delete	Access Control	PASS	—
TC-05	Mass Assignment on Profile Update	Input Validation	PASS	—
TC-06	Admin Login Brute-Force & Rate-Limiting	Authentication	OBSERVATION	Medium
TC-07	Session Cookie Security Attributes	Configuration	FAIL	Low
TC-08	HTTP → HTTPS Enforcement	Configuration	PASS	—

6. Detailed Test Cases

TC-01 — Perimeter & Service Exposure Scan

Module / Endpoint: Network / Infrastructure

Category: Infrastructure security

Priority: High

Preconditions: Target domain is resolvable; tester has standard internet access to the target's public-facing IP.

Steps to Reproduce

1. Resolve the target application's public IP address.
2. Run: nmap -sT -Pn -p 22,80,443,3306,5432,8080 [Target_IP]
3. Send a plain HTTP request to port 80 and observe whether it is redirected.
4. Send a request with a manipulated/spoofed Host header directly to the resolved IP and observe the response.

Test Data: Port list: 22, 80, 443, 3306, 5432, 8080; arbitrary Host header value

Expected Result

Management and database ports (22, 3306, 5432) should be closed or filtered from the public internet. Plain HTTP (port 80) should redirect to HTTPS. Requests with a mismatched Host header should not reach the application logic.

Actual Result

Ports 22, 3306, and 5432 returned a filtered state; raw connection attempts on port 80 were dropped immediately (tcpwrapped). A spoofed Host header over HTTP/2 returned 421 Misdirected Request, confirming that virtual-host isolation is strictly enforced.

Status: **PASS**

TC-02 — Sensitive Data Exposure in Listing-Creation API Response

Module / Endpoint: POST /api/panel/listing

Category: API / Sensitive data exposure

Priority: Medium

Preconditions: Tester holds a valid, authenticated standard-user (non-admin) session.

Steps to Reproduce

1. Authenticate as a standard user and capture the session cookie.
2. Send a POST request to /api/panel/listing with a valid JSON body and the session cookie attached.
3. Capture the complete JSON response body returned by the server.
4. Inspect the response for any fields beyond the expected operation status and resulting object ID.

Test Data: Request body:
{"title":"UIDCROSSACCESSTEST","description":"","phone_wp":"0532 000 00 00"}

Expected Result

The response should contain only the operation status and the resulting listing ID. No internal user role, permission, or privilege data should be returned to the client.

Actual Result

Request sent:

```
POST /api/panel/listing HTTP/1.1
Host: [REDACTED SUBDOMAIN]
Cookie: session=[REDACTED]
Content-Type: application/json
Content-Length: 75

{"title":"UIDCROSSACCESSTEST","description":"","phone_wp":"0532 000 00 00"}
```

Response received:

```
{
  "status": "success",
  "listing_id": "7110c5d6-7142-4940-8d99-dbe9064164a9",
  "user": {
    "id": "usr_94a8c2...",
    "role": "user",
    "is_admin": false,
    "privileges": []
  }
}
```

The response includes a nested user object exposing role and privilege data ("is_admin", "role", "privileges") that is unrelated to the listing-creation operation and was not expected to be returned to the client.

Status: FAIL | Risk: Medium

TC-03 — Stored Content Injection via SVG File Upload

Module / Endpoint: POST /api/panel/listing (image/file upload)

Category: Input validation / Stored injection

Priority: High

Preconditions: Tester has an authenticated session able to create a listing with an image attachment; an externally controlled request-logging endpoint is available for verification.

Steps to Reproduce

1. Craft an SVG file whose <image> element references an external, tester-controlled request-logging URL.
2. Encode the SVG file as Base64.
3. Submit the Base64-encoded SVG as the image payload of a listing-creation request.

4. Open/preview the created listing through the admin panel or listing preview screen.
5. Monitor the external logging endpoint for inbound requests.

Test Data: SVG payload referencing an external image URL (listener ID redacted for this report)

Expected Result

The server should validate the uploaded file's signature (magic bytes) and reject or strip SVG content containing external references before persisting it.

Actual Result

SVG payload used:

```
<?xml version="1.0" standalone="no"?>
<svg width="640px" height="480px" version="1.1" xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink">
  <image xlink:href="https://webhook.site/[REDACTED LISTENER ID]"
    x="0" y="0" height="640px" width="480px"/>
</svg>
```

The server accepted and permanently stored the payload without validating its file signature. When the listing was rendered in the admin preview screen, the rendering browser resolved the embedded external reference and the logging endpoint recorded successful inbound requests, confirming the injection executes on view (Stored HTML/Image Injection). No script-signature or anomaly-based filtering was observed on the upload pipeline; were the payload to be obfuscated further, the same gap could plausibly be escalated toward script execution in the admin session.

Status: FAIL | Risk: Medium

TC-04 — Object-Level Authorization (IDOR / BOLA) on Listing Deletion

Module / Endpoint: POST /api/panel/listing/delete

Category: Access control

Priority: Critical

Preconditions: Two distinct authenticated accounts are available: Account A (owns a listing) and Account B (unrelated).

Steps to Reproduce

1. As Account A, create a listing and capture its listing ID.
2. As Account B, authenticate and capture a valid session cookie.
3. Using Account B's session, send a delete request referencing Account A's listing ID.
4. Repeat the same request with no session cookie attached.
5. Record the HTTP status code returned in both cases.

Test Data: Account A listing ID: 7110c5d6-7142-4940-8d99-dbe9064164a9; Account B session cookie (redacted)

Expected Result

The server should reject the delete request because Account B does not own the target object, regardless of holding a valid session.

Actual Result

Request sent:

```
POST /api/panel/listing/delete HTTP/1.1
Host: [REDACTED SUBDOMAIN]
Cookie: session=[REDACTED]
Content-Type: application/json
Content-Length: 53

{"id": "7110c5d6-7142-4940-8d99-dbe9064164a9"}
```

The server returned 401 Unauthorized for both the cross-account attempt and the unauthenticated attempt, confirming that object ownership is validated server-side before any delete operation is performed.

Status: PASS

TC-05 — Mass Assignment / Privilege Escalation via Profile Update

Module / Endpoint: POST/PUT profile-update endpoint(s)

Category: Input validation / Authorization

Priority: High

Preconditions: Tester holds a valid authenticated standard-user session.

Steps to Reproduce

1. Authenticate as a standard user.
2. Send a profile-update request containing legitimate fields plus an injected "is_admin": true field.
3. Inspect the server's response.
4. Re-check the account's effective role/permissions after the request completes.

Test Data: Injected field: "is_admin": true

Expected Result

The server should ignore any field not defined in its update contract; the account's role should remain unchanged.

Actual Result

The injected is_admin field had no observable effect — the account's role and permissions remained unchanged after the request, confirming the endpoint uses a whitelisted Data Transfer Object (DTO) rather than binding the raw request body.

Status: PASS

TC-06 — Admin Login Brute-Force & Rate-Limiting Behavior

Module / Endpoint: Admin login / passcode endpoint

Category: Authentication

Priority: Medium

Preconditions: A candidate wordlist was built from character/numeric patterns observed in client-side resources.

Steps to Reproduce

1. Configure an automated request tool (Burp Intruder) to submit the candidate list sequentially against the admin login endpoint.
2. Monitor responses for blocking behavior (HTTP 429, IP ban) during the first several dozen attempts.
3. Continue submitting attempts for at least 15 minutes and observe whether the candidate pool's validity resets.

Test Data: Automated passcode wordlist; sequential submission via Burp Intruder

Expected Result

The endpoint should apply rate-limiting (e.g. HTTP 429) or a progressive lockout after a small number of failed attempts.

Actual Result

No IP blocking or 429 response was observed during the early attempts. A business-logic-level control was identified, however: a 15-minute active window after which the candidate pattern pool is invalidated and reset by the backend. This limits, but does not eliminate, brute-force feasibility — a dedicated rate-limit control is recommended.

Status: OBSERVATION | Risk: Medium

TC-07 — Session Cookie Security Attribute Validation

Module / Endpoint: Authentication / Session management

Category: Configuration

Priority: Medium

Preconditions: Valid login credentials are available.

Steps to Reproduce

1. Log in to the application.
2. Inspect the Set-Cookie header returned for the session cookie.
3. Verify the presence of the HttpOnly, Secure, and SameSite attributes.

Test Data: session=[REDACTED]

Expected Result

The session cookie should be issued with HttpOnly, Secure, and an appropriate SameSite policy.

Actual Result

HttpOnly and SameSite=Lax were present and correctly configured. The Secure attribute was missing, meaning the cookie could be transmitted unencrypted if the client is ever directed to a plain-HTTP connection.

Status: FAIL | Risk: Low

TC-08 — HTTP → HTTPS Enforcement & Cleartext Transport Check

Module / Endpoint: Transport layer / web server configuration

Category: Configuration

Priority: Medium

Preconditions: Ability to issue plain HTTP requests to the main application and at least one sub-storefront subdomain.

Steps to Reproduce

1. Send a request over plain HTTP (port 80) to the main application.
2. Repeat the request against a sub-storefront subdomain.
3. Observe whether each connection is redirected to HTTPS or served in cleartext.
4. If a non-Secure session cookie is active, check whether it is transmitted in a cleartext request.

Expected Result

All HTTP requests, across the main application and every subdomain/storefront, should be redirected to HTTPS without exception.

Actual Result

An earlier pass found that at least one sub-storefront permitted direct cleartext HTTP access, which — combined with the missing Secure cookie flag (TC-07) — exposed the session value on the local network. On retest, the server now rejects or redirects all HTTP traffic to HTTPS, closing the previously observed leak path. This historical finding is retained here for traceability.

Status: PASS

7. Recommendations

1. **REC-01 (relates to TC-03):** Harden the SVG/file upload pipeline by validating file signatures (magic bytes) and stripping or rejecting embedded external references before persisting uploaded files.
2. **REC-02 (relates to TC-02):** Sanitize API response payloads so that internal role/permission objects are never returned to the client.
3. **REC-03 (relates to TC-07):** Add the Secure flag to all session cookies.
4. **REC-04 (relates to TC-06):** Implement a formal rate-limiting control (HTTP 429) on the admin authentication endpoint, in addition to the existing time-bound pattern reset.

8. Conclusion

The platform demonstrates strong network hardening and a well-implemented authorization layer: object-level access control (TC-04) and mass-assignment protection (TC-05) both passed without exception, and no Critical or High-severity issue capable of full system compromise was confirmed. The three failed cases (TC-02, TC-03, TC-07) are all Medium/Low risk individually, but TC-03 in particular should be prioritized, since it is the only case where a working proof-of-concept was reproduced end-to-end. Addressing the four recommendations above would bring all eight test cases to a passing state.